SECURE CODING STANDARD

OWASP Top 10-aligned application security requirements

Control	Value
Document owner	Product owner / engineering lead
Status	Draft for organizational adoption
Version	1.0
Effective date	Upon approval and publication
Review cadence	At least annually and after material architecture or threat changes
Applies to	WC01 web, API, MCP, worker, infrastructure-as-code, CI/CD, and supporting operational code

Questionnaire use: Select “Yes” for OWASP Top 10 secure-coding practices only after the accountable owner adopts this standard, developers are notified, and the required pull-request and scanning evidence is retained.

1. Purpose and policy statement

CertScore.ai incorporates security into design, implementation, review, testing, deployment, and maintenance. This standard defines the minimum secure-coding practices used to address common vulnerability classes, including the OWASP Top 10. It is a control standard and operating checklist; it is not a certification or a guarantee that every vulnerability is prevented.

All production changes must be traceable to a reviewed pull request, pass applicable automated checks, and be evaluated for security impact before deployment. Security defects are prioritized using the vulnerability-management SLAs in the security operations procedure.

2. Scope and responsibilities

- Product owner: approves adoption, accepts residual risk, assigns remediation owners, and reviews exceptions.
- Change author: applies this standard, records security-relevant design decisions, adds tests, and responds to review findings.
- Independent reviewer: checks security impact, tests, authorization, data handling, dependency changes, and rollback implications before approval.
- Operations/release owner: confirms CI results, deployment protections, secret handling, and post-deployment monitoring.

3. Secure-coding principles

- Least privilege: grant the minimum identity, permission, network, database, and tool access needed for the task.
- Fail closed: reject malformed, unauthenticated, unauthorized, stale, unverifiable, or insufficiently evidenced input and state.
- Validate at trust boundaries: parse and validate URLs, headers, tokens, identifiers, JSON, file paths, and external responses before use.
- Encode and parameterize: use parameterized database access, context-appropriate output encoding, safe URL handling, and approved serializers.
- Minimize data: do not log secrets, tokens, raw credentials, payment-card data, unbounded request/response bodies, or unnecessary personal data.
- Make security observable: emit privacy-minimized security events, retain actionable audit records, and alert on material failures.
- Use defense in depth: combine application controls, dependency checks, infrastructure controls, secure configuration, and monitoring.

4. OWASP Top 10 control mapping

ID	Vulnerability class	Minimum required practice
A01	Broken Access Control	Enforce authorization on every protected route and tool; bind decisions to authenticated subject, tenant, role, and resource; deny by default; test IDOR and cross-tenant access.
A02	Cryptographic Failures	Use TLS for transport; use approved encryption and managed secret stores; never commit secrets; avoid exposing tokens or sensitive values in logs, URLs, errors, or analytics.
A03	Injection	Use parameterized queries and safe SDK calls; validate and constrain shell, URL, template, HTML, SQL, and JSON inputs; treat scanner content and model output as untrusted data.
A04	Insecure Design	Threat-model new trust boundaries, data flows, authorization changes, external integrations, and high-impact workflows; document abuse cases and safe failure behavior.
A05	Security Misconfiguration	Use secure defaults, explicit allowlists, hardened headers, private storage, least-privilege IAM, disabled debug paths, dependency pinning, and configuration review.
A06	Vulnerable and Outdated Components	Run dependency and infrastructure vulnerability scans; review advisories; patch according to severity SLAs; record mitigations and risk acceptance when a fix is not immediately available.
A07	Identification and Authentication Failures	Use approved authentication flows; validate issuer, audience, tenant, signature, expiry, token type, and required roles; protect sessions and account recovery; never accept role-less protected requests.
A08	Software and Data Integrity Failures	Require reviewed changes, protected branches, reproducible builds where feasible, signed or immutable deployment artifacts, dependency integrity checks, and verification of retained evidence.
A09	Security Logging and Monitoring Failures	Log security-relevant outcomes without secrets; review logs on a defined cadence; alert for authentication, authorization, availability, and integrity anomalies; preserve incident evidence.

ID	Vulnerability class	Minimum required practice
A10	Server-Side Request Forgery	Allow outbound requests only to validated, intended destinations; block private/link-local metadata targets; enforce scheme, DNS, redirect, timeout, response-size, and content-type limits.

5. Required development workflow

1. Design review: identify trust boundaries, identities, sensitive data, external services, authorization decisions, and abuse cases before implementation.
2. Implementation: use approved libraries and patterns; validate untrusted input; apply least privilege; keep secrets and sensitive values out of code and logs.
3. Automated checks: run unit/integration security tests, dependency checks, infrastructure-as-code checks, lint/type checks, and relevant static analysis before merge.
4. Independent review: a reviewer other than the author verifies security impact, access control, input handling, data exposure, tests, and rollback implications.
5. Release: deploy only from the protected production workflow after required checks and approvals pass; use immutable image or artifact references where available.
6. Post-release: monitor security events and errors, investigate anomalies, and record corrective actions or risk acceptance.

6. Pull-request security checklist

- ☐ Does the change alter authentication, authorization, tenant isolation, secrets, payment, privacy, or external network behavior?
- ☐ Are all new inputs validated for type, length, format, encoding, and allowed values?
- ☐ Are database, shell, template, URL, HTML, and deserialization operations parameterized or safely constrained?
- ☐ Are permissions, roles, IAM actions, API scopes, and network destinations least-privilege and deny-by-default?
- ☐ Could logs, errors, telemetry, screenshots, artifacts, or model prompts expose secrets or personal data?
- ☐ Are negative tests included for unauthorized, malformed, cross-tenant, stale, expired, and oversized inputs?
- ☐ Are dependencies, containers, Terraform, and runtime configuration checked for known vulnerabilities or insecure defaults?
- ☐ Is a reviewer other than the author assigned, and are findings resolved or documented as accepted risk?

7. Testing and evidence requirements

The following records must be retained for production changes when applicable:

- Pull request URL, author, reviewer, approval, changed scope, and security-impact assessment.
- Test results covering authentication, authorization, input validation, negative paths, tenant isolation, and sensitive-data handling.
- Dependency, container, and infrastructure scan results, including remediation issue or documented risk acceptance for unresolved findings.
- Deployment artifact identifier, environment, approval, and rollback reference.
- Security incident, alert, or post-release review record when the change affects a material security control.

8. Vulnerability handling and exceptions

Security vulnerabilities are triaged and remediated under the production patching SLAs: critical or known exploited issues within 72 hours, high within 14 calendar days, medium within 30 calendar days, and low within 90 calendar days. A mitigation may replace a patch only when it is documented and demonstrably reduces exposure.

An exception requires: the affected control, reason, scope, risk assessment, compensating control, named owner, expiration/review date, and product-owner approval. Exceptions do not authorize logging secrets, bypassing tenant isolation, accepting unauthenticated protected access, or weakening evidence integrity.

9. Adoption and review

This standard becomes an operating control when the accountable owner signs below, publishes it to the engineering team, and links it from the production change workflow. Review it at least annually, after a material security incident, and before a material change to authentication, authorization, data processing, external integrations, or deployment architecture.

Adoption item	Record
Accountable owner	_____
Approval date	_____
Publication location	_____
Next review date	_____
Evidence location	_____

Questionnaire answer: After adoption and evidence retention, the organization may answer “Yes” to: “Do secure coding practices take into account common vulnerability classes such as OWASP Top 10?” Before adoption, answer “No.”

Appendix A. References

- OWASP Top 10 (2021): https://owasp.org/Top10/2021/
- NIST Secure Software Development Framework (SSDF): https://csrc.nist.gov/Projects/ssdf
- CertScore security operations procedure: /Users/benmasek/WC01/docs/security-operations.md
- CertScore vulnerability register: /Users/benmasek/WC01/docs/security-vulnerability-register.md